

Bimodal Battery Optimization

Bimodal Capacity Optimization | 1. Prior Research on Bimodal Packing Density

Open Access Review

A Review of Particle Packing Models and Their Applications to Characterize Properties of Sand-Silt Mixtures

by Ching S. Chang * and Jason Chao

Department of Civil and Environmental Engineering, University of Massachusetts, Amherst, MA 01003, USA

* Author to whom correspondence should be addressed.

Geotechnics **2024**, *4*(4), 1124–1139; <https://doi.org/10.3390/geotechnics4040057>

Submission received: 7 September 2024 / Revised: 3 October 2024 / Accepted: 29 October 2024 /

Published: 1 November 2024

In this project, the size ratio r (small/large particles)
 $= 4\mu\text{m}/8\mu\text{m} = 0.5$ follows the non-linear model
($0.741 > 0.5 > 0.154$)



3D Simulation Attempt

The model considers three conditions.

For $r \geq 0.741$ (near uniform packing), $\beta_{12} = \gamma_{12} = 0$, representing the upper bound.

For $0.741 > r > 0.154$, non-linear behavior exhibits with values of β_{12} and γ_{12} determined by the fitting method suggested in [29]. The binary mixture of particles is mainly characterized by occupation (mixing effect).

For $r < 0.154$, where the small particle size is less than the cavity size, it simplifies to the linear packing model [17], characterized by filling (unmixing effects).

At $r = 0.154$, a slight discontinuity in the estimated packing densities occurs when transitioning between the linear and non-linear models.

The linear packing models for predicting the packing behavior of binary mixtures typically rely on two straight lines, one representing the coarse-grain dominant region and the other for the fine-grain dominant region. However, when the particle size ratio $r > 0.22$, these models often fail to accurately predict experimental results due to the non-linearity of the v vs. y_2 curve. This non-linearity becomes especially pronounced in transitional regions where neither coarse nor fine particles dominate entirely.

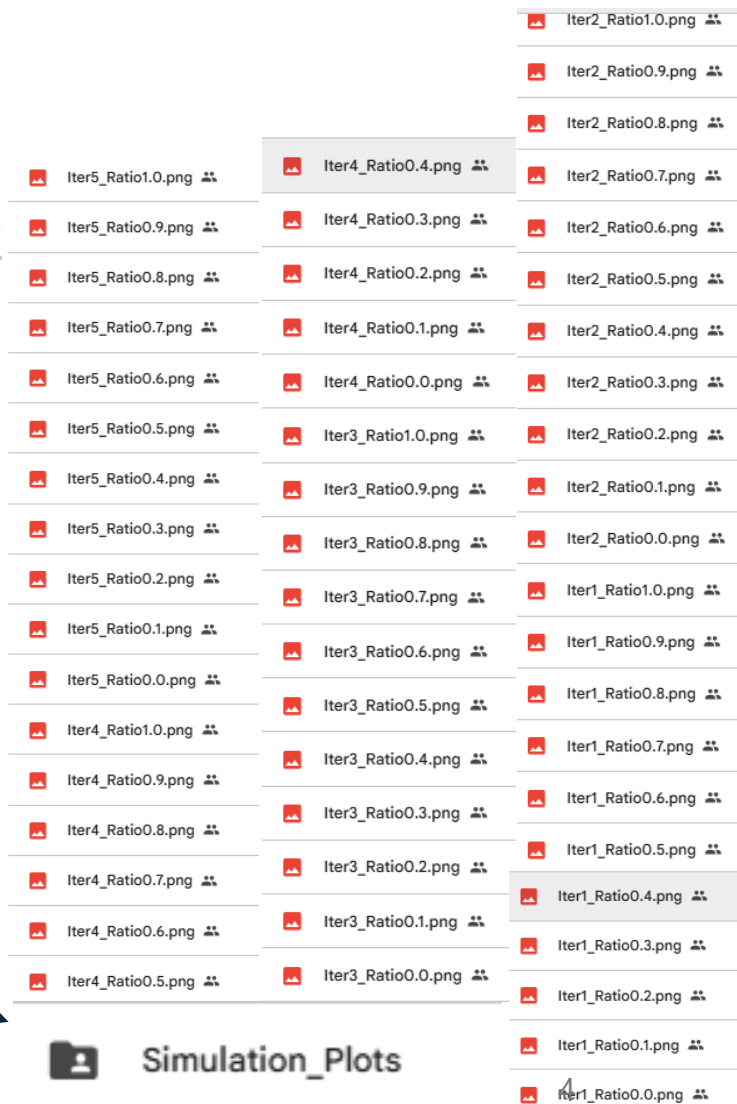
It is noted that all of the non-linear functions $\Delta v(r, y_1, y_2)$ discussed here are phenomenological constructs. Using a single function of Δv for the entire range of fines content would result in a highly complex function. Since the packing structure can be broadly

Bimodal Capacity Optimization | 2. 3D Simulation Code Explanation

Small particle
diameter: $4\mu\text{m}$
Large particle
diameter: $8\mu\text{m}$
Particles are spherical.

```
1 clear; clc; close all;
2 %% 1. 설정 파라미터
3 { d_small = 4;
4   d_large = 8;
5   L = 100;
6   L_sub = L / 3;
7   sub_min = L / 3;
8   sub_max = 2 * L / 3;
9   SubVol = L_sub^3;
10  % 입자 재료 밀도 (g/cc)
11  material_density = 4.81756;
12  % 비율 및 반복 횟수 설정
13  ratios = linspace(0, 1, 11); % 0.1 간격, 11개 지점
14  num_sims = length(ratios);
15  num_iterations = 5; % 5번 반복
16  % 결과 저장 형식 (질량 밀도 저장용)
17  all mass results = zeros(num_iterations, num_sims);
18  % 이미지 저장 폴더 생성
19  plot_folder = 'Simulation_Plots';
20  if ~exist(plot_folder, 'dir')
21      mkdir(plot_folder);
22  end
```

Small : Large particle ratio
Total 11 ratios from 0.0 to 1.0
in 0.1 steps
(Small particle basis: 0.0, 0.1,
0.2,...1.0)



Bimodal Capacity Optimization | 2. 3D Simulation Code Explanation

<Particle Stacking Rules>

Find cc/cc (particle space occupancy ratio)

Minimum spacing is
Small - Small: 4um
Large - Large: 8um
Small - Large: 6um

Stack particles in order of
lowest center height
coordinate while satisfying
minimum spacing
requirements.

Multiply cc/cc (particle volume fraction) by g/cc (bulk material density) to obtain g/cc (effective material density in space)

```
24 % 2. 반복 시뮬레이션 루프
25 for iter = 1:num_iterations
26     for s = 1:num_sims
27         vol_ratio = ratios(s);
28         fprintf('실행 중: 반복[%d/%d] - 비율[%1f]...\n', iter, num_iterations, vol_ratio);
29         particles = [];
30         fail_count = 0;
31         max_fails = 50;
32         % [입자 생성 및 배치 로직]
33         while fail_count < max_fails
34             if isempty(particles)
35                 is_small = (vol_ratio > 0);
36             elseif vol_ratio == 0, is_small = false;
37             elseif vol_ratio == 1, is_small = true;
38             else
39                 small_indices = (abs(particles(:,4) - d_small/2) < 1e-6);
40                 curr_v_s = sum((4/3) * pi * particles(small_indices, 4).^3);
41                 curr_v_total = sum((4/3) * pi * particles(:, 4).^3);
42                 is_small = (curr_v_s / curr_v_total) < vol_ratio;
43             end
44             d_target = ifthen(is_small, d_small, d_large);
45             r_target = d_target / 2;
46             best_z = 1e6; best_pos = [];
47             for trial = 1:30
48                 tx = r_target + (L - d_target)*rand();
49                 ty = r_target + (L - d_target)*rand();
50                 tz = r_target;
51                 for i = 1:size(particles, 1)
52                     dx = tx - particles(i,1); dy = ty - particles(i,2);
53                     dist_2d = sqrt(dx^2 + dy^2);
54                     r_other = particles(i,4);
55                     % 기존 get_min_dist 대신 직접 계산 (속도 향상)
56                     if is_small && abs(r_other - d_small/2) < 1e-6, min_dist = d_small;
57                     elseif ~is_small && abs(r_other - d_small/2) >= 1e-6, min_dist = d_large;
58                     else, min_dist = (d_small + d_large)/2;
59                     end
60                     if dist_2d < min_dist
61                         h = particles(i,3) + sqrt(min_dist^2 - dist_2d^2);
62                         tz = max(tz, h);
63                     end
64                 end
65                 if tz + r_target <= L && tz < best_z
66                     best_z = tz;
67                     best_pos = [tx, ty, tz, r_target];
68                 end
69             end
70         end
71     end
72 end
```

```
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
end

if ~isempty(best_pos)
    particles = [particles; best_pos];
    fail_count = 0;
else
    fail_count = fail_count + 1;
end
end

% [부피 및 질량 밀도 계산]
sub_total_vol = 0;
for i = 1:size(particles, 1)
    pr = particles(i,4);
    if (particles(i,1)+pr > sub_min && particles(i,1)-pr < sub_max && ...
        particles(i,2)+pr > sub_min && particles(i,2)-pr < sub_max && ...
        particles(i,3)+pr > sub_min && particles(i,3)-pr < sub_max)
        v_part = calculatePartialSphereVolume(particles(i,1), particles(i,2), particles(i,3), pr, sub_min, sub_max);
        sub_total_vol = sub_total_vol + v_part;
    end
end

% 공간 점유율(cc/cc)에 물질 밀도(g/cc)를 곱하여 저장
packing_fraction = sub_total_vol / subvol;
all_mass_results(iter, s) = packing_fraction * material_density;

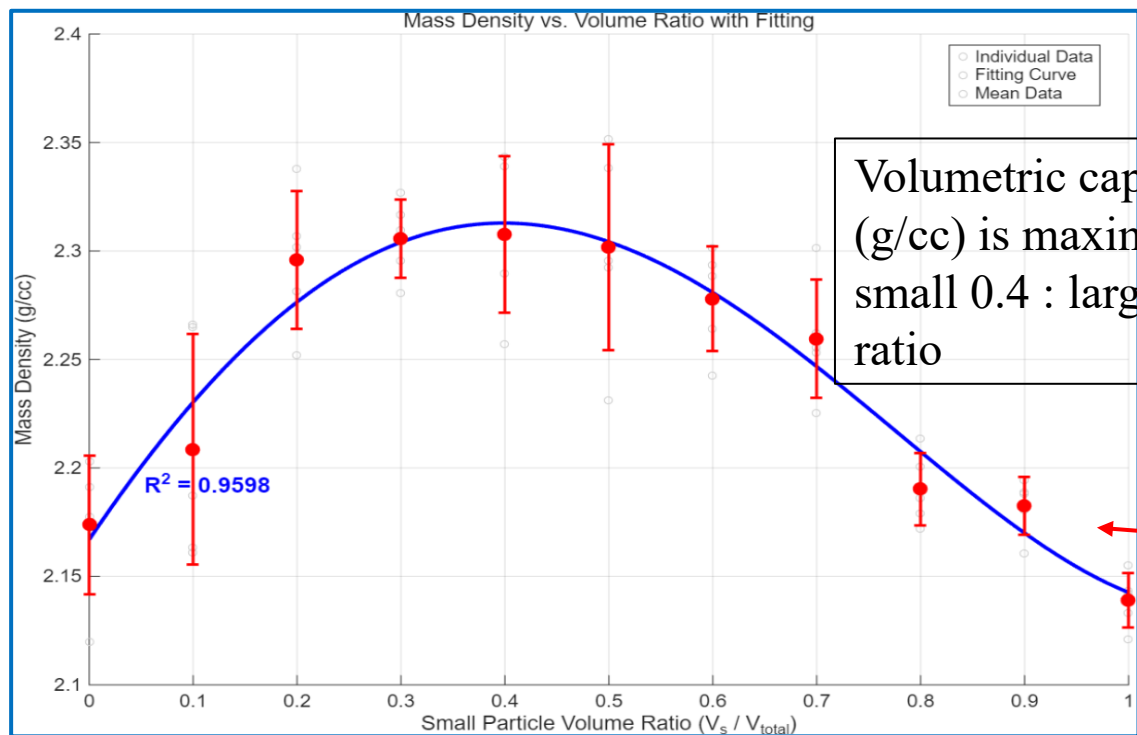
% [3D 시각화 및 이미지 저장]
fig_3d = figure('Visible', 'off', 'Renderer', 'painters'); % Renderer 추가로 안정성 향상
hold on;
[sx, sy, sz] = sphere(12);
for i = 1:size(particles, 1)
    px = particles(i,1); py = particles(i,2); pz = particles(i,3); pr = particles(i,4);
    c = ifthen(pr < d_large/2, [0.3 0.6 1], [1 0.4 0.4]); % 스:파랑, 대:빨강
    surf(sx*pr + px, sy*pr + py, sz*pr + pz, 'FaceColor', c, 'EdgeColor', 'none');
end
axis equal; view(3); grid on; light; lighting gouraud;
title(sprintf('Iter %d, Ratio %1f', iter, vol_ratio));
xlabel('X'); ylabel('Y'); zlabel('Z');
xlim([0 L]); ylim([0 L]); zlim([0 L]);
drawnow; % 렌더링 강제
pause(0.01); % 그래픽스 파이프라인에 약간의 시간 부여
print(fig_3d, fullfile(plot_folder, sprintf('Iter%d_Ratio%1f.png', iter, vol_ratio)), '-dpng'); % saveas 대신 print 사용
close(fig_3d);
end
```

Bimodal Capacity Optimization | 2. 3D Simulation Code Explanation

	A	B	C	D	E	F	G	H	I	J	K
1	Ratio_0.0	Ratio_0.1	Ratio_0.2	Ratio_0.3	Ratio_0.4	Ratio_0.5	Ratio_0.6	Ratio_0.7	Ratio_0.8	Ratio_0.9	Ratio_1.0
2	2.177619751	2.187442948	2.301625133	2.316493125	2.256937909	2.338287304	2.288422089	2.225487027	2.172042173	2.160554026	2.155076345
3	2.202793776	2.160803769	2.251776567	2.280397036	2.339186376	2.295414874	2.301541885	2.30139204	2.200396249	2.18804233	2.142222941
4	2.177486555	2.266211673	2.337854417	2.326749209	2.289737399	2.231097904	2.242486154	2.255322908	2.21369919	2.188791557	2.143255209
5	2.119679535	2.264829766	2.281179562	2.295681266	2.308900959	2.292384667	2.293483533	2.262049301	2.17895171	2.194369135	2.121061442
6	2.191205732	2.163217944	2.306819773	2.30950034	2.343398696	2.351673492	2.264130487	2.253091877	2.18607769	2.180749854	2.133315465

Save g/cc by ratio as CSV file

Plot



Volumetric capacity (g/cc) is maximized at small 0.4 : large 0.6 ratio

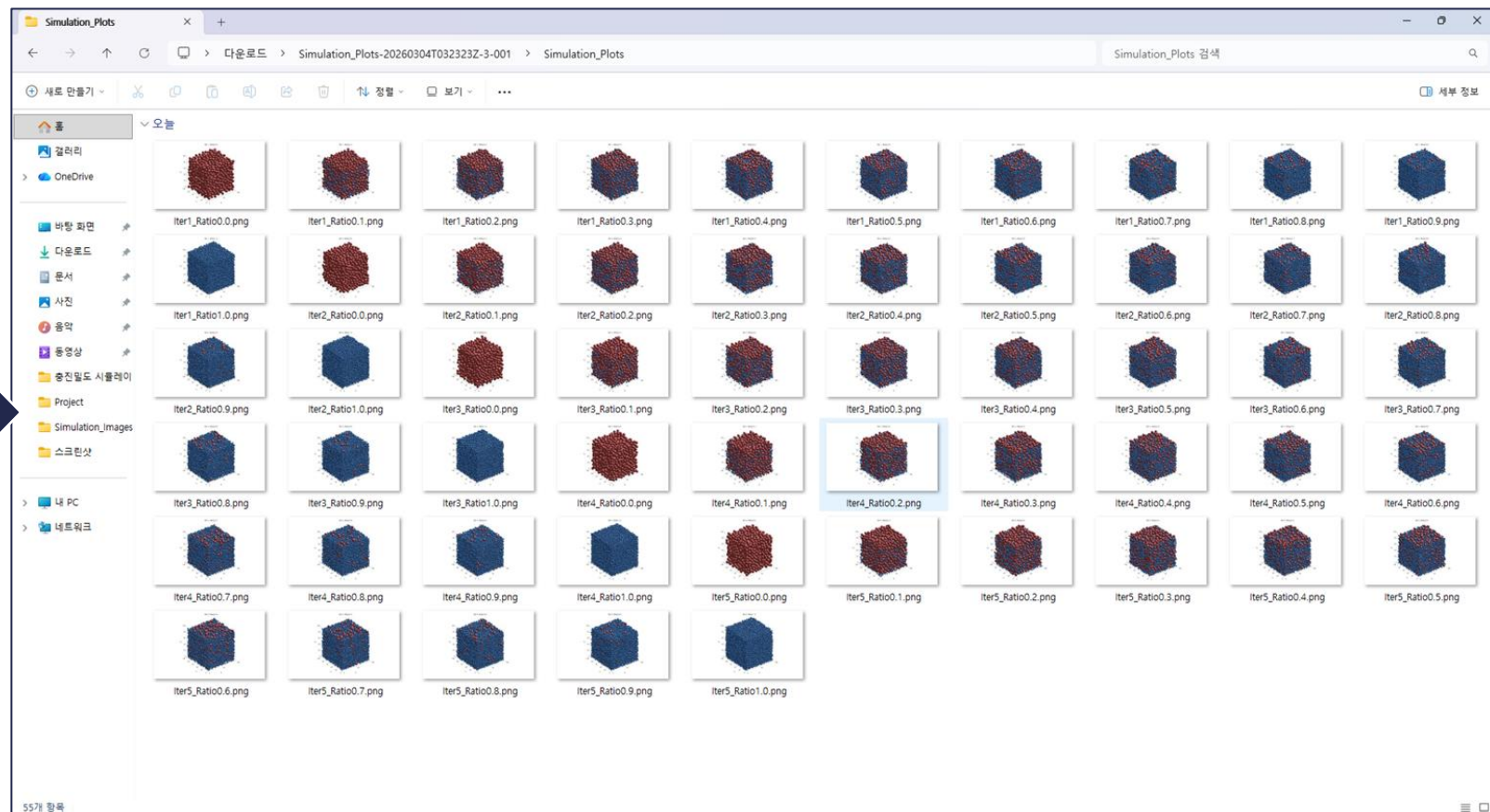
```

109 %% 3. 데이터 저장 (CSV)
110 column_names = cell(1, num_sims);
111 for i = 1:num_sims
112     column_names{i} = sprintf('Ratio_%.1f', ratios(i));
113 end
114 T = array2table(all_mass_results, 'VariableNames', column_names);
115 writetable(T, 'Mass_Density_Results.csv');
116 %% 4. 결과 시각화 및 피팅 (Fitting)
117 figure('Color', 'w', 'Position', [100, 100, 900, 600]);
118 hold on;
119 % 평균값 및 표준편차 계산
120 mean_mass_densities = mean(all_mass_results, 1);
121 plot(ratios, all_mass_results, 'o', 'Color', [0.8 0.8 0.8], 'Markersize', 4); % 개별 점
122 % 다항식 피팅 (4차식 - 비선형 추세 반영에 적합)
123 p = polyfit(ratios, mean_mass_densities, 4);
124 fit_x = linspace(0, 1, 100);
125 fit_y = polyval(p, fit_x);
126 % R^2 계산
127 y_pred = polyval(p, ratios);
128 S_res = sum((mean_mass_densities - y_pred).^2);
129 S_tot = sum((mean_mass_densities - mean(mean_mass_densities)).^2);
130 R2 = 1 - (S_res / S_tot);
131 % 그래프 그리기
132 plot(fit_x, fit_y, 'b-', 'Linewidth', 2, 'DisplayName', 'Trend Line (4th Poly)');
133 errorbar(ratios, mean_mass_densities, std(all_mass_results, 0, 1), 'ro', ...
134         'Linewidth', 1.5, 'MarkerFaceColor', 'r', 'DisplayName', 'Mean ± STD');
135 grid on;
136 xlabel('Small Particle Volume Ratio ( $V_s / V_{total}$ )');
137 ylabel('Mass Density (g/cc)');
138 title('Mass Density vs. Volume Ratio with Fitting');
139 legend('Individual Data', 'Fitting Curve', 'Mean Data', 'Location', 'Best');
140 % R^2 값 텍스트 박스 추가
141 text(0.05, max(mean_mass_densities)*0.95, sprintf('R^2 = %.4f', R2), ...
142     'FontSize', 12, 'FontWeight', 'bold', 'Color', 'b');
    
```

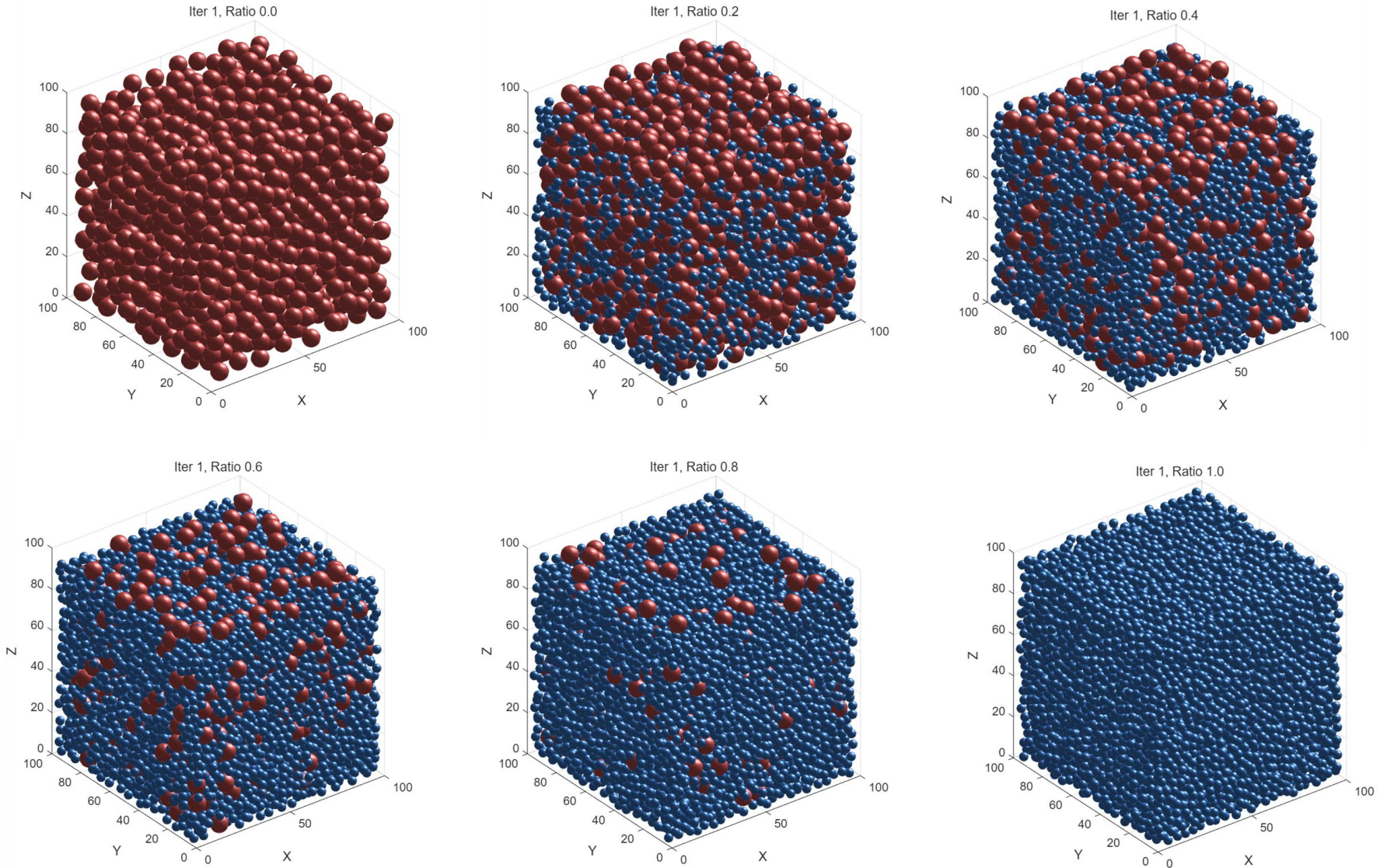
Bimodal Capacity Optimization | 3. 3D Simulation Process

명령 창

```
진행 중: 반복[1/5] - 비율[0.0]...
진행 중: 반복[1/5] - 비율[0.1]...
진행 중: 반복[1/5] - 비율[0.2]...
진행 중: 반복[1/5] - 비율[0.3]...
진행 중: 반복[1/5] - 비율[0.4]...
진행 중: 반복[1/5] - 비율[0.5]...
진행 중: 반복[1/5] - 비율[0.6]...
진행 중: 반복[1/5] - 비율[0.7]...
진행 중: 반복[1/5] - 비율[0.8]...
진행 중: 반복[1/5] - 비율[0.9]...
진행 중: 반복[1/5] - 비율[1.0]...
진행 중: 반복[2/5] - 비율[0.0]...
진행 중: 반복[2/5] - 비율[0.1]...
진행 중: 반복[2/5] - 비율[0.2]...
진행 중: 반복[2/5] - 비율[0.3]...
진행 중: 반복[2/5] - 비율[0.4]...
진행 중: 반복[2/5] - 비율[0.5]...
진행 중: 반복[2/5] - 비율[0.6]...
진행 중: 반복[2/5] - 비율[0.7]...
진행 중: 반복[2/5] - 비율[0.8]...
진행 중: 반복[2/5] - 비율[0.9]...
진행 중: 반복[2/5] - 비율[1.0]...
진행 중: 반복[3/5] - 비율[0.0]...
진행 중: 반복[3/5] - 비율[0.1]...
진행 중: 반복[3/5] - 비율[0.2]...
진행 중: 반복[3/5] - 비율[0.3]...
진행 중: 반복[3/5] - 비율[0.4]...
진행 중: 반복[3/5] - 비율[0.5]...
진행 중: 반복[3/5] - 비율[0.6]...
진행 중: 반복[3/5] - 비율[0.7]...
진행 중: 반복[3/5] - 비율[0.8]...
진행 중: 반복[3/5] - 비율[0.9]...
진행 중: 반복[3/5] - 비율[1.0]...
진행 중: 반복[4/5] - 비율[0.0]...
진행 중: 반복[4/5] - 비율[0.1]...
진행 중: 반복[4/5] - 비율[0.2]...
진행 중: 반복[4/5] - 비율[0.3]...
진행 중: 반복[4/5] - 비율[0.4]...
진행 중: 반복[4/5] - 비율[0.5]...
진행 중: 반복[4/5] - 비율[0.6]...
진행 중: 반복[4/5] - 비율[0.7]...
진행 중: 반복[4/5] - 비율[0.8]...
진행 중: 반복[4/5] - 비율[0.9]...
진행 중: 반복[4/5] - 비율[1.0]...
진행 중: 반복[5/5] - 비율[0.0]...
진행 중: 반복[5/5] - 비율[0.1]...
진행 중: 반복[5/5] - 비율[0.2]...
진행 중: 반복[5/5] - 비율[0.3]...
진행 중: 반복[5/5] - 비율[0.4]...
진행 중: 반복[5/5] - 비율[0.5]...
진행 중: 반복[5/5] - 비율[0.6]...
진행 중: 반복[5/5] - 비율[0.7]...
진행 중: 반복[5/5] - 비율[0.8]...
진행 중: 반복[5/5] - 비율[0.9]...
진행 중: 반복[5/5] - 비율[1.0]...
>>
```



Bimodal Capacity Optimization | 3. 3D Simulation Process



Bimodal Capacity Optimization | 4. Optimization of Ratio for Power and Capacity

Specific Power by Small Particle Ratio

Volumetric Capacity by Small Particle Ratio

	A	B
1	Small_Particle_Ratio	Predicted_30s_Specific_Power_W_kg
2	0	6719.897619
3	0.02	6825.959027
4	0.04	6932.020435
5	0.06	7038.081842
6	0.08	7144.14325
50	0.96	11810.84518
51	0.98	11916.90659
52	1	12022.968

	A	B
1	Small_Particle_Ratio	Packing_Density
2	0	2.167
3	0.02	2.180900413
4	0.04	2.194203253
5	0.06	2.206882621
6	0.08	2.218914874
50	0.96	2.151745474
51	0.98	2.14681903
52	1	2.1426

```
%% 1. 데이터 로드
density_table = readtable('Mass_Density_Data.csv');
power_table = readtable('Specific_Power_30s_Trend_Data.csv');
```

```
% 데이터 추출
ratio = density_table.Small_Particle_Ratio;
val_density = density_table.Packing_Density;
val_power = power_table.Predicted_30s_Specific_Power_W_kg;
```

Each 51 data points

```
%% 2. 정규화 (Min-Max Scaling to 50 Points)
norm_density = (val_density - min(val_density)) / (max(val_density) - min(val_density)) * 50;
norm_power = (val_power - min(val_power)) / (max(val_power) - min(val_power)) * 50;
total_score = norm_density + norm_power;
```

```
%% 3. 주요 지점 계산 (최대 밀도, 최대 출력, 최적 합산)
```

```
% 1) 최대 밀도(Capacity) 지점
```

```
[max_dens_val, idx_dens] = max(norm_density);
ratio_dens = ratio(idx_dens);
```

```
% 2) 최대 출력(Power) 지점
```

```
[max_pow_val, idx_pow] = max(norm_power);
ratio_pow = ratio(idx_pow);
```

```
% 3) 최적 합산(Total) 지점
```

```
[max_total_val, idx_total] = max(total_score);
ratio_total = ratio(idx_total);
```

Determine the maximum **Total Score** to identify the optimal point that achieves the best balance between two performance metrics

Min-Max Normalization (Feature Scaling)

Min-max normalization performs a linear transformation on the original data. Suppose that min_A and max_A are the minimum and maximum values of an attribute, A . Min-max normalization maps a value, v_i , of A to v'_i in the range $[new_min_A, new_max_A]$ by computing

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Bimodal Capacity Optimization | 4. Optimization of Ratio for Power and Capacity

```
%% 4. CSV 파일로 결과 저장
result_table = table(ratio, norm_density, norm_power, total_score, ...
    'VariableNames', {'Small_Particle_Ratio', 'Density_Score_50pt', 'Power_Score_50pt', 'Total_Score_100pt'});
writetable(result_table, 'Combined_Score_Results.csv');
fprintf('결과가 "Combined_Score_Results.csv"로 저장되었습니다.\n');
```

```
%% 5. 시각화 (그래프화)
figure('Color', 'w', 'Name', 'Optimal Design Point Analysis');
grid on; hold on;
```

```
% 개별 요소 및 합산 점수 플롯
p1 = plot(ratio, norm_density, '--', 'LineWidth', 1.2, 'Color', [0 0.447 0.741]);
p2 = plot(ratio, norm_power, '--', 'LineWidth', 1.2, 'Color', [0.466 0.674 0.188]);
p3 = plot(ratio, total_score, 'k-', 'LineWidth', 2.5);
```

```
% --- 추가 지점 표시 (강조가 적은 색상: Gray/DarkGray) ---
% 최대 밀도 지점 표시
plot(ratio_dens, max_dens_val, 'o', 'Color', [0.5 0.5 0.5], 'MarkerSize', 6);
text(ratio_dens, max_dens_val - 3, sprintf('Max Density\n (Ratio: %.2f)', ratio_dens), ...
    'FontSize', 8, 'Color', [0.3 0.3 0.3], 'HorizontalAlignment', 'center');
```

```
% 최대 출력 지점 표시
plot(ratio_pow, max_pow_val, 's', 'Color', [0.5 0.5 0.5], 'MarkerSize', 6);
text(ratio_pow, max_pow_val - 3, sprintf('Max Power\n (Ratio: %.2f)', ratio_pow), ...
    'FontSize', 8, 'Color', [0.3 0.3 0.3], 'HorizontalAlignment', 'center');
```

```
% --- 최적 합산 지점 표시 (기준 강조색: Red) ---
plot(ratio_total, max_total_val, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
text(ratio_total, max_total_val + 4, sprintf('★ Optimal Combined\n Ratio: %.2f (Score: %.1f)', ratio_total, max_total_val), ...
    'FontWeight', 'bold', 'FontSize', 10, 'Color', 'r');
```

```
% 그래프 꾸미기
xlabel('Small Particle Ratio (소립 비율)', 'FontSize', 11);
ylabel('Normalized Score', 'FontSize', 11);
title('소립 비율별 설계 지점 분석 (밀도 vs 출력)', 'FontSize', 13);
legend([p1, p2, p3], {'Mass Density Score', 'Specific Power Score', 'Total Score'}, ...
    'Location', 'southoutside', 'Orientation', 'horizontal');
ylim([0 110]); % 텍스트 표시 공간 확보
hold off;
```

```
%% 6. 결과 출력 (커맨드 창)
fprintf('-----\n');
fprintf('1. 최대 밀도 설계 지점: Ratio %.2f\n', ratio_dens);
fprintf('2. 최대 출력 설계 지점: Ratio %.2f\n', ratio_pow);
fprintf('3. 최적 합산 설계 지점: Ratio %.2f (총점: %.2f)\n', ratio_total, max_total_val);
fprintf('-----\n');
```

